

CO1-ASSESSMENT TOOL-1

Name: TALARI VISHNUVARDHAN

Reg.no: 192572091

Course name: DESIGN AND ANALYSIS OF ALGORITHMS

Code: CSA0613

Faculty: Dr. F. MARY HARIN FERNANDEZ

Date: 16-07-2026

Question 38

Title : Application: Image Processing System

Recurrence Relation : $T(n) = 4T(n/2) + n$

Problem Statement

An image processing algorithm divides images recursively: $T(n)=4T(n/2)+n$. Apply the Master Theorem to find the time complexity. Clearly identify parameters and case selection. Analyze how performance changes with image size.

Objective

- Analyze the recurrence using the Master Theorem.
- Identify a, b and f(n).
- Determine the time complexity.
- Analyze performance as image size increases.

Concept Used

Master Theorem for $T(n)=aT(n/b)+f(n)$.

Algorithm / Approach

1. Identify a, b and f(n).
2. Compute $n^{(\log_b a)}$.
3. Select the Master Theorem case.
4. Determine the time complexity.
5. Analyze scalability.

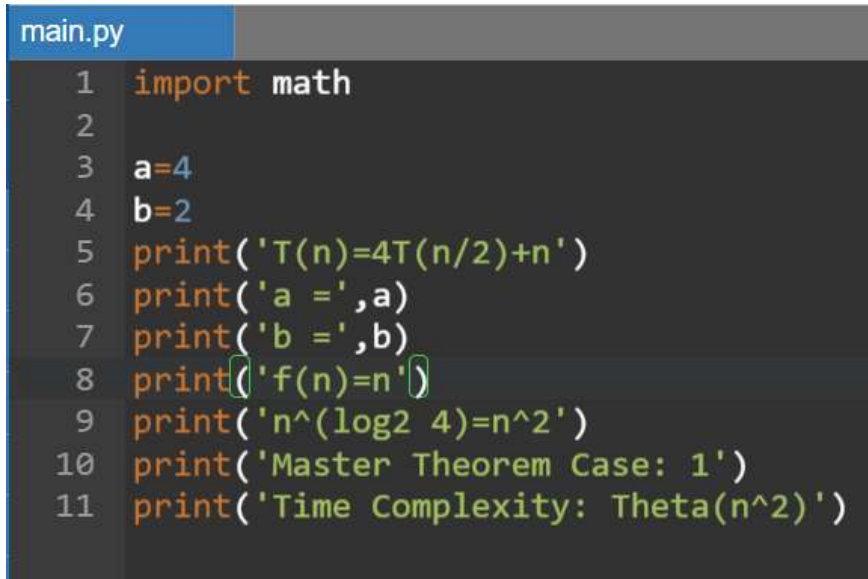
Source Code (Python)

```
import math
```

```
a=4
```

```
b=2
print('T(n)=4T(n/2)+n')
print('a =',a)
print('b =',b)
print('f(n)=n')
print('n^(log2 4)=n^2')
print('Master Theorem Case: 1')
print('Time Complexity: Theta(n^2)')
```

Source Code Screenshot

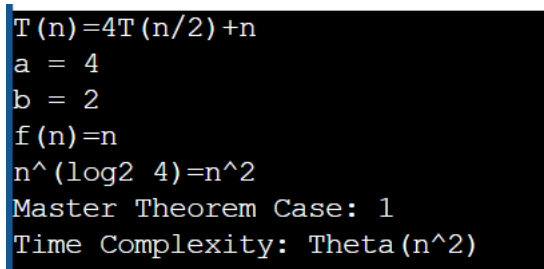


```
main.py
1  import math
2
3  a=4
4  b=2
5  print('T(n)=4T(n/2)+n')
6  print('a =',a)
7  print('b =',b)
8  print('f(n)=n')
9  print('n^(log2 4)=n^2')
10 print('Master Theorem Case: 1')
11 print('Time Complexity: Theta(n^2)')
```

Sample Output

```
T(n)=4T(n/2)+n
a = 4
b = 2
f(n)=n
n^(log2 4)=n^2
Master Theorem Case: 1
Time Complexity: Theta(n^2)
```

Output Screenshot



```
T (n)=4T (n/2) +n
a = 4
b = 2
f (n)=n
n^(log2 4)=n^2
Master Theorem Case: 1
Time Complexity: Theta (n^2)
```

Analysis

$a=4$, $b=2$, $f(n)=n \cdot n^{(\log_2 4)}=n^2$. Since $f(n)=O(n^{(2-1)})$, Master Theorem Case 1 applies. Therefore $T(n)=\Theta(n^2)$.

Performance Analysis

- Image is divided into four sub-images.
- Time grows as $\Theta(n^2)$.
- Doubling image size increases work about 4x.
- Large images require more computation.
- Efficient for moderate image sizes.

Conclusion

The recurrence satisfies Master Theorem Case 1. Therefore the time complexity is $\Theta(n^2)$.

Question 33

Title: Application: Social Media Feed Ranking

Recurrence: $T(n) = 4T(n/2) + n^2$

Problem Statement

A social media platform ranks posts using $T(n)=4T(n/2)+n^2$. Apply the Master Theorem to compute the time complexity. Clearly specify parameters and case selection. Analyze scalability issues as the number of posts increases and interpret efficiency.

Objective

- Analyze the recurrence using the Master Theorem.
- Determine the time complexity.
- Study scalability as the number of posts increases.

Concept Used

Master Theorem for $T(n)=aT(n/b)+f(n)$.

Algorithm / Approach

1. Identify a, b and f(n).
2. Compute $n^{(\log_b a)}$.
3. Select the Master Theorem case.
4. Derive the time complexity.
5. Interpret scalability.

Implementation (Source Code)

```
# Python Program
import math
a=4
b=2
print("Recurrence: T(n)=4T(n/2)+n^2")
print("a =",a)
print("b =",b)
print("f(n)=n^2")
print("n^(log24)=n^2")
print("Master Theorem Case: 2")
print("Time Complexity:  $\Theta(n^2 \log n)$ ")
```

```

main.py
1  # Python Program
2  import math
3  a=4
4  b=2
5  print("Recurrence: T(n)=4T(n/2)+n^2")
6  print("a =",a)
7  print("b =",b)
8  print("f(n)=n^2")
9  print("n^(log_2 4)=n^2")
10 print("Master Theorem Case: 2")
11 print("Time Complexity: O(n^2 log n)")

```

Sample Output

```

Recurrence: T(n)=4T(n/2)+n^2
a = 4
b = 2
f(n)=n^2
n^(log_2 4)=n^2
Master Theorem Case: 2
Time Complexity: O(n^2 log n)

```

```

Recurrence: T(n)=4T(n/2)+n^2
a = 4
b = 2
f(n)=n^2
n^(log_2 4)=n^2
Master Theorem Case: 2
Time Complexity: O(n^2 log n)

```

Analysis

Parameters: $a=4$, $b=2$, $f(n)=n^2$.

$n^{(\log_2 4)}=n^2$, therefore $f(n)=\Theta(n^{(\log_2 a)})$. Hence Master Theorem Case 2 applies.

Final Time Complexity: $\Theta(n^2 \log n)$.

Scalability and Efficiency

- As the number of posts increases, execution time grows as $\Theta(n^2 \log n)$.
- Doubling the posts increases computation by more than four times.
- Suitable for small and medium datasets.
- Large datasets require more computation and memory.
- Parallel processing can improve scalability.

Conclusion

The recurrence satisfies Master Theorem Case 2. Therefore, the time complexity is $\Theta(n^2 \log n)$. The algorithm is efficient for moderate datasets but becomes expensive for very large numbers of posts.

Programming Language : Python 3

IDE Used : Visual Studio Code

Algorithm Used : Master Theorem

Time Complexity : $\Theta(n^2 \log n)$

Date of Execution : (16-7-2026)

Author : Talari Vishnuvardhan